

React Component Lifecycle

1. Mounting (Component created)

Runs once after the component is rendered:

```
useEffect(() => {  
  console.log("Mounted");  
}, []); // empty dependency array → runs once
```

✓ Use cases:

- API calls
- DOM setup
- Subscriptions

2. Updating

Runs when dependencies change:

```
useEffect(() => {  
  console.log("Updated due to count change");  
}, [count]); // runs every time `count` changes
```

✓ Use cases:

- Respond to state/prop changes
- Trigger side effects when a variable changes

3. 3. Unmounting

Cleanup before the component is removed:

```
useEffect(() => {  
  const timer = setInterval(() => console.log("tick"), 1000);  
  return () => clearInterval(timer); // cleanup on unmount  
}, []); // empty array → cleanup runs once
```

✓ Use cases:

- Clear timers

- Cancel API requests
- Remove event listeners

Class Based components Life cycle

Mount → Update → Unmount → Error

- Mount = setup
- Update = re-render
- Unmount = cleanup
- Error = fallback handling

1. Mounting – When a component is created and inserted into the DOM

- `constructor(props)` – Initialize state and bind methods.
- `static getDerivedStateFromProps(props, state)` – Sync state with props (rarely used).
- `render()` – Returns JSX to display. Pure function, no side effects.
- `componentDidMount()` – Called after mounting. Ideal for API calls, DOM interactions, subscriptions.

2. Updating – When a component re-renders due to state or props changes

- `static getDerivedStateFromProps(props, state)` – Update state based on props.
- `shouldComponentUpdate(nextProps, nextState)` – Decide if re-render is necessary (performance optimization).
- `render()` – Re-render the component.
- `getSnapshotBeforeUpdate(prevProps, prevState)` – Capture DOM info before it changes.
- `componentDidUpdate(prevProps, prevState, snapshot)` – Runs after DOM updates. Useful for post-update actions like API calls.

3. Unmounting – When a component is removed from the DOM

- `componentWillUnmount()` – Cleanup timers, subscriptions, event listeners.

4. Error Handling (Optional)

- `static getDerivedStateFromError(error)` – Update state to show fallback UI.
- `componentDidCatch(error, info)` – Log errors or perform side effects.

Reconciliation

Reconciliation is the process by which React updates the DOM efficiently when a component's state or props change. Instead of re-rendering the entire DOM, React compares the previous and new virtual DOM and applies only the minimal necessary changes to the real DOM.

How Reconciliation Works

1. Virtual DOM creation – React keeps a lightweight virtual DOM representing the UI in memory.
2. Diffing – When a state/prop changes, React creates a new virtual DOM and compares it with the previous virtual DOM.
3. Patch/Update – React calculates the minimum set of changes and updates the real DOM accordingly.

✅ This is more efficient than updating the real DOM directly.

React uses three main heuristics to optimize reconciliation:

1. Element Type

- If two elements are of different types, React destroys the old DOM node and creates a new one.
- Example: `<div>` replaced by `<span>` → full replacement.

2. Keys in Lists

- Keys help React identify elements in arrays/lists.
- Example: `<li key={id}>`
- Without keys, React may re-render unnecessarily or reorder incorrectly.
- Keys should be stable, unique, and predictable.

3. Component Type

- If the component type is the same, React reuses the DOM node and only updates changed props/state.
- If the component type changes, React destroys the old tree and mounts a new one.

3. When Reconciliation Happens

- On state change (setState or useState)
- On prop change
- On parent component re-render

State/Props Change



Virtual DOM Update



Diffing Algorithm

(prevVDOM vs nextVDOM)



Minimal DOM Updates



Real DOM Updated

Q: What optimization methods exist for reconciliation?

A: React.memo, shouldComponentUpdate, proper use of keys, immutable data structures.

Re-render Triggers

A re-render occurs when React decides a component needs to update its UI based on changes in state, props, or context.

1. State Changes

- Updating a component's state via:

`this.setState({ key: value }); // class component`
`setState(value); // functional component`

- React schedules a re-render for that component.

✓ Example:

```
const [count, setCount] = useState(0);  
setCount(count + 1); // triggers re-render
```

2. Props Changes

- When parent passes new props to a child component, it triggers a re-render.
- React does a shallow comparison of props (unless optimized).

✓ Example:

```
<ChildComponent value={parentState} /> // updates when parentState  
changes
```

3. Context Changes

- When a component consumes React Context, it re-renders if the context value changes.
- Use `React.memo` or `useContextSelector` for optimization.

4. Force Update (Rare / Advanced)

- **Class Components:** `this.forceUpdate()` forces a re-render without state change.
- **Functional Components:** Usually achieved via state toggling.

5. Parent Re-renders

- By default, every time a parent re-renders, all children re-render as well.
- **Optimization tools:**
 - `React.memo` → memoize functional components
 - `PureComponent` → shallow prop/state comparison in class components

Lifting state

Lifting state up is a React pattern where you move state from a child component to the closest common parent so that multiple child components can share and synchronize data.

- React state is local to the component it's defined in.
- When two or more components need the same state, you lift the state to their nearest common ancestor.
- This avoids duplication and keeps the state single source of truth.
- Promotes unidirectional data flow: parent → children via props, children → parent via callbacks.

When to Use Lifting State Up

- Two child components need to share data.
- Child components need to communicate or react to the same state

3. Practical Example

```
import React, { useState } from "react";

// Parent
function Parent() {
  const [text, setText] = useState("");

  return (
    <div>
      <ChildA value={text} onChange={setText} />
      <ChildB value={text} />
    </div>
  );
}

// ChildA (Input)
function ChildA({ value, onChange }) {
  return (
    <input
      value={value}
      onChange={(e) => onChange(e.target.value)}
    />
  );
}

// ChildB (Display)
```

```
function ChildB({ value }) {  
  return <h2>{value}</h2>;  
}
```

```
export default Parent;
```

Flow in Words (Very Simple)

1. Parent stores the state
2. Parent sends value to ChildA & ChildB
3. User types in ChildA
4. ChildA calls parent callback
5. Parent updates state
6. Parent re-renders
7. Both children receive updated value

Prop Drilling

Prop drilling is when you pass data from a parent to a deeply nested child through intermediate components that don't actually use the data.

Parent



ChildA



ChildB



ChildC (needs the data)

Ultra Basic Code Example

```
import React, { useState } from "react";
```

```
function Parent() {  
  const [message] = useState("Hello from Parent");  
  
  return <ChildA message={message} />;  
}  
  
function ChildA({ message }) {  
  return <ChildB message={message} />;  
}  
  
function ChildB({ message }) {  
  return <ChildC message={message} />;  
}  
  
function ChildC({ message }) {  
  return <h2>{message}</h2>;  
}  
  
export default Parent;
```

What's the Problem?

- ChildA doesn't need message
- ChildB doesn't need message
- But they must accept and pass it down
- Becomes messy in deep trees

When Is It a Problem?

- Deep component trees
- Too many shared values
- Hard to maintain
- Refactoring becomes painful

What Solves It?

- Context API
- State management libraries (Redux, Zustand, etc.)

Interview Questions

1 Why does parent re-render cause child re-render?

When a parent re-renders, React re-executes its function, which recreates its children elements.

By default, React re-renders child components unless they are memoized (React.memo, PureComponent).

Re-render $\neq$ DOM update — React may skip DOM changes after diffing.

2 Why does useEffect run twice in StrictMode?

In development, React StrictMode intentionally double-invokes effects (mount $\rightarrow$ unmount $\rightarrow$ mount) to detect:

- Side effects that aren't properly cleaned up
- Unsafe lifecycle behavior

This happens only in development, not production.

3 Why do inline functions cause re-renders?

Inline functions create a new function reference on every render.

If passed as props to a child, shallow comparison fails → child re-renders.

Solution: useCallback for stable reference (when necessary).

4 How does diffing work in $O(n)$?

React uses heuristics:

1. Different element types → replace subtree.
2. Same type → compare props.
3. Lists rely on stable key to match elements.

Instead of deep tree comparison ($O(n^3)$), React assumes:

- Same type = same structure.
- Keys identify stable elements.

Thus it achieves linear $O(n)$ complexity.

5 Why does single source of truth matter?

Because duplicated state causes inconsistency.

Keeping state in one place ensures:

- Predictable updates
- Easier debugging
- No synchronization bugs

React's unidirectional data flow depends on it.

6 Architectural thinking behind state placement

State should live at the lowest common ancestor of components that need it.

Guidelines:

- If only one component needs it → keep it local.
- If siblings need it → lift it up.
- If many distant components need it → consider Context or global store.

Goal: Minimize prop drilling and unnecessary re-renders.

7 Why do effects run after paint?

useEffect runs after the browser paints to avoid blocking UI rendering.

React:

- 1. Renders UI**
- 2. Commits DOM**
- 3. Browser paints**
- 4. Then runs effects**

This keeps UI responsive.

8 Race conditions in async effects

Race conditions occur when multiple async requests overlap and resolve out of order.

Example:

- **User types fast**
- **First request resolves after second**
- **Old data overrides new data**

Solution:

- **Track active request**
- **Use AbortController**
- **Or cleanup logic inside effect**

Fetch Lifecycle

In React, fetch is typically triggered inside `useEffect` after the component mounts. The lifecycle includes initial render, effect execution, loading state, success/error state updates, and cleanup. Advanced considerations include race conditions, stale closures, StrictMode double invocation, and aborting requests to prevent memory leaks.

When fetching data inside a React component, the lifecycle typically follows this sequence:

1 Initial Render (Before Fetch)

- Component renders.
- State is usually:
 - `loading = true`
 - `data = null`
 - `error = null`

No API call has completed yet.

2 Effect Runs (After Paint)

`useEffect` runs after the component mounts.

React flow:

1. Render phase
2. Commit phase
3. Browser paint
4. `useEffect` executes
5. Fetch starts

Important: Fetch is triggered after paint, not during render.

3 Fetch In Progress

While fetch is pending:

- Component may re-render due to other state changes.
- You must avoid memory leaks if component unmounts.

At this stage:

- Loading UI is shown.

4 Success Case

When fetch resolves:

- `setData(response)`
- `setLoading(false)`

This triggers a re-render:

- New virtual DOM
- Reconciliation
- DOM updates

5 Error Case

If fetch fails:

- `setError(error)`
- `setLoading(false)`

Triggers re-render to show error UI.

6 Cleanup Phase (Critical for 3-Year Level)

If component unmounts before fetch completes:

- Updating state causes memory leak warning.
- You must cancel or ignore stale request.

Solution:

- **AbortController**
- **Or track isMounted flag inside effect**

Proper fetch lifecycle includes:

- **Loading state**
- **Error handling**
- **Abort on unmount**
- **Race condition handling**
- **Dependency correctness**

Race Conditions in Fetch (Interview-Ready, Brief)

Problem:

When a dependency changes quickly:

- **Request A starts**
- **Request B starts**
- **B finishes first → state updates correctly**
- **A finishes later → overwrites state with stale data**

This causes outdated UI.

Why It Happens

Because async calls resolve independently, and React doesn't automatically cancel previous requests.

Solutions

1 Abort Previous Request (Preferred)

Use `AbortController` inside `useEffect` cleanup to cancel old fetch when dependency changes or component unmounts.

2 Track Latest Request

Store a request ID or flag.

Only update state if the response matches the latest request.

Interview One-Liner

Race conditions happen when multiple async requests resolve out of order, causing stale data to override fresh data. We prevent this by aborting previous requests or validating the latest request before updating state.

React Forms

forms are handled using controlled or uncontrolled components.

React prefers controlled components because state becomes the single source of truth.

In controlled components, form state is managed by React, making it predictable and easier to validate.

Uncontrolled components rely on the DOM for state management and are accessed using refs.

◆ Handling Multiple Inputs

Use dynamic keys:

```
onChange={(e) =>
  setForm({ ...form, [e.target.name]: e.target.value })
}
```

Shows understanding of:

- Immutable updates
- Dynamic state handling

◆ Form Submission

- Always prevent default:

```
onSubmit={(e) => {  
  e.preventDefault();  
}}
```

- Validation before API call
- Handle loading & error states

◆ Performance Considerations

Large forms can cause:

- Many re-renders
- Laggy typing

Solutions:

- Debouncing
- useReducer
- Form libraries (React Hook Form, Formik)
- Uncontrolled inputs for heavy forms

◆ Common Pitfalls

- Forgetting value → switching between controlled/uncontrolled
- Infinite re-render from state updates inside render
- Not handling async validation properly

In React, forms are typically implemented using controlled components where state lives in React, ensuring predictable updates and easier validation. For large or performance-critical forms, uncontrolled components or form libraries can reduce re-renders and improve efficiency.

Higher Order Components

A Higher Order Component is a function that takes a component and returns a new enhanced component. It's used to reuse cross-cutting concerns like authentication or logging. While hooks have largely replaced HOCs in modern React, they're still important in legacy and library-level patterns.

An HOC:

- Receives a component
- Adds extra behavior (logic, props, side effects)
- Returns a new component

Ultra Basic Example

```
function withLogger(WrappedComponent) {  
  return function Enhanced(props) {  
    console.log("Component rendered");  
    return <WrappedComponent {...props} />;  
  };  
}
```

Usage:

```
const LoggedComponent = withLogger(MyComponent);
```

Now LoggedComponent has logging behavior.

Important HOC Rules

1 Don't Mutate the Original Component

Always return a new component.

Bad:

```
WrappedComponent.prototype.newMethod = ...
```

Good:

Return a new wrapper component.

2 Pass Props Properly

Always spread props:

```
<WrappedComponent {...props} />
```

Otherwise you block parent props.

Real-World Example (Auth HOC Conceptually)

withAuth(Component)

- Checks if user is authenticated
- If yes → render component
- If not → redirect to login

HOC vs Hooks

HOC

Pattern for class components

Wraps component

Can cause wrapper nesting

Harder debugging sometimes

Hooks

Built for functional components

Uses function composition

Cleaner logic reuse

More readable

Modern React prefers **Hooks over HOCs**, but HOCs are still used in:

- Legacy codebases
- Libraries (e.g., Redux connect())